

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 - FLOWCHART-TO-CODE CONVERSION

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO5 - Naturalization (BL4, BL6)	Assessment:	Assessment Tool 2 - Flowchart-To-Code Conversion
Weightage:	35% of CIA	Total Marks:	100
CO5 Statement:	Construct MST using shortest path algorithms and traverse the graph to model and analyse real-world problem.		
Student Name:	A. Lkithasri	Reg. No.:	192511003
Section / Batch:	1	Date:	02/09/2026

Code of Conduct

I, A. Lkithasri (192511003) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Lkithasri

Instructions

- This test contains 5 Flowchart-to-code conversion questions. Total: 100 Marks.
- When a student answer using Flowchart-to-code conversion should evaluate based on the ability to design clear, logical, and efficient code solutions for data structure problems.
- No negative marking. Unanswered questions carry zero marks.

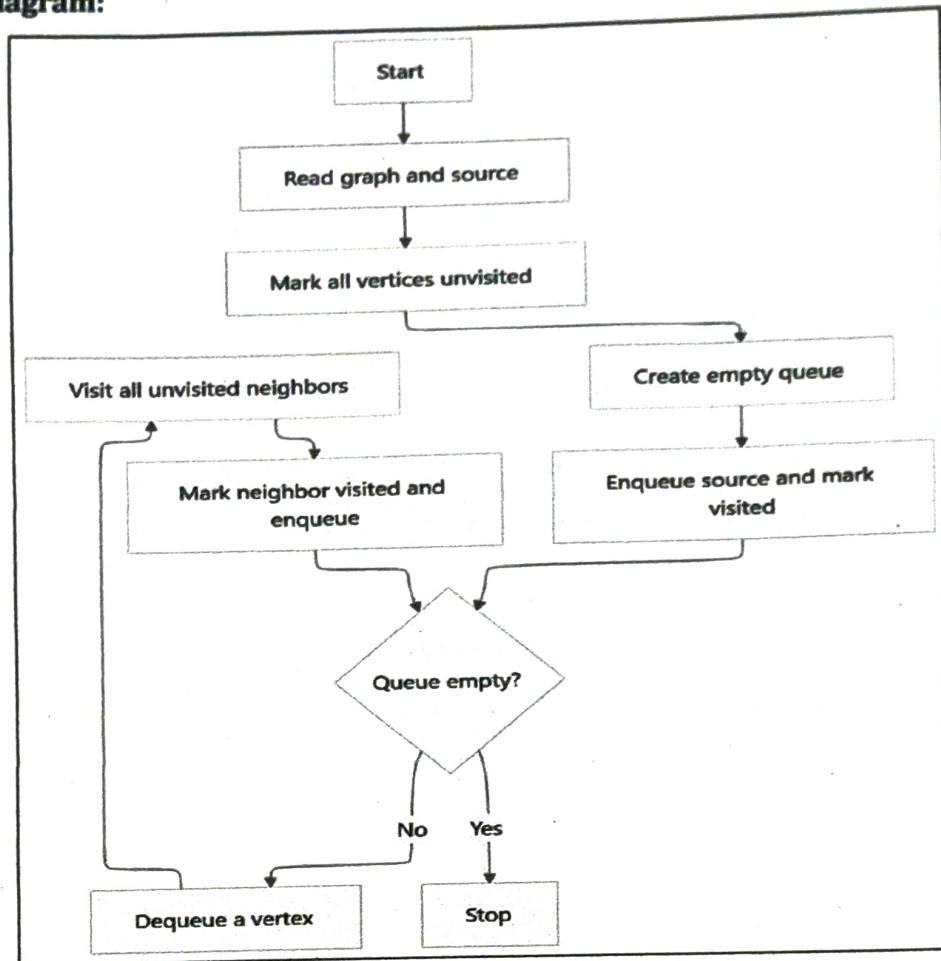
Section / Topic	Focus	Q Nos	Marks
Graph Traversal	BFS	1	20
Minimum Spanning Tree	Kruskal's Algorithm	2	20
Graph Sorting	Topological Sort	3	20
Graph Traversal	DFS	4	20
Minimum Spanning Tree	Prim's Algorithm	5	20
GRAND TOTAL:			100 Marks

Annexure A - Flowchart-To-Code Conversion

(20 Marks)

1. Convert the flowchart for Breadth First Search traversal into code.

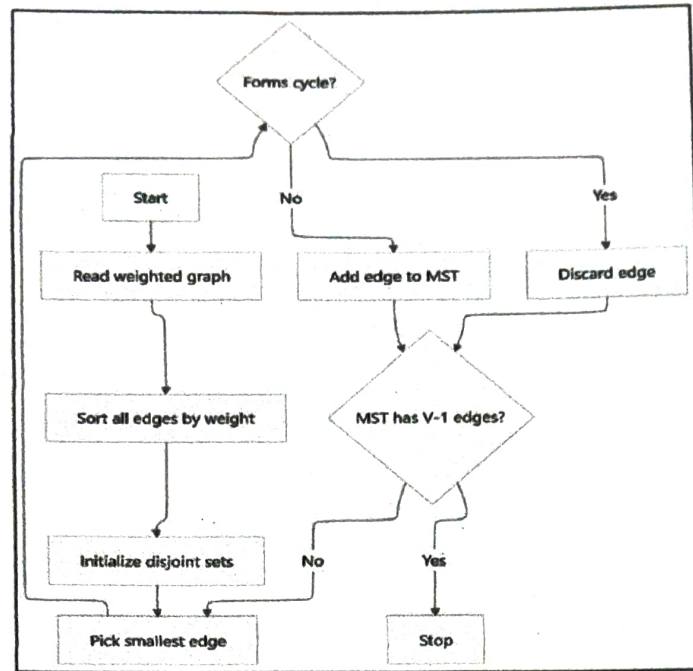
Flowchart Diagram:



2. Convert the flowchart for Kruskal's algorithm into code.

(20 Marks)

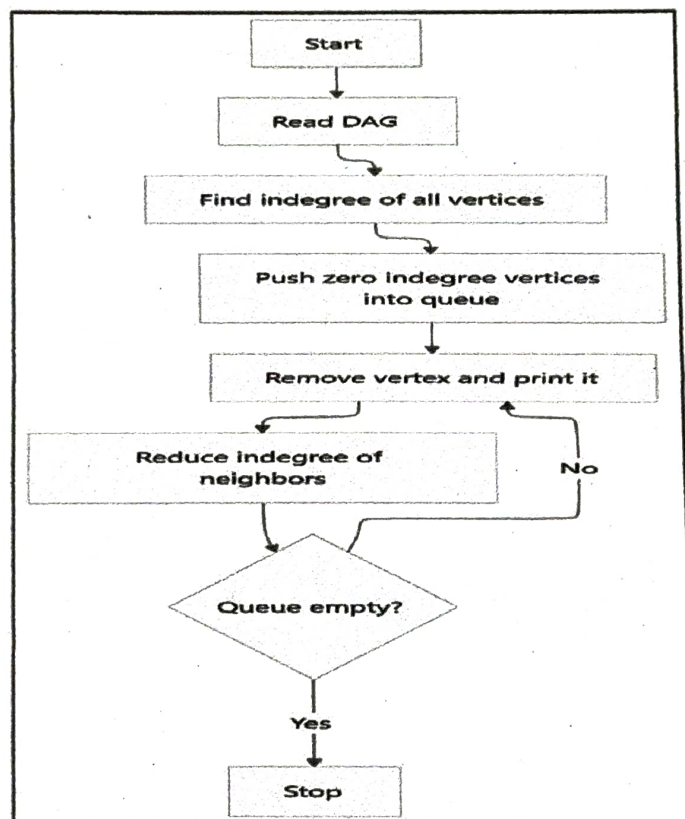
Flowchart Diagram:



3. Convert the flowchart for topological sort into code.

(20 Marks)

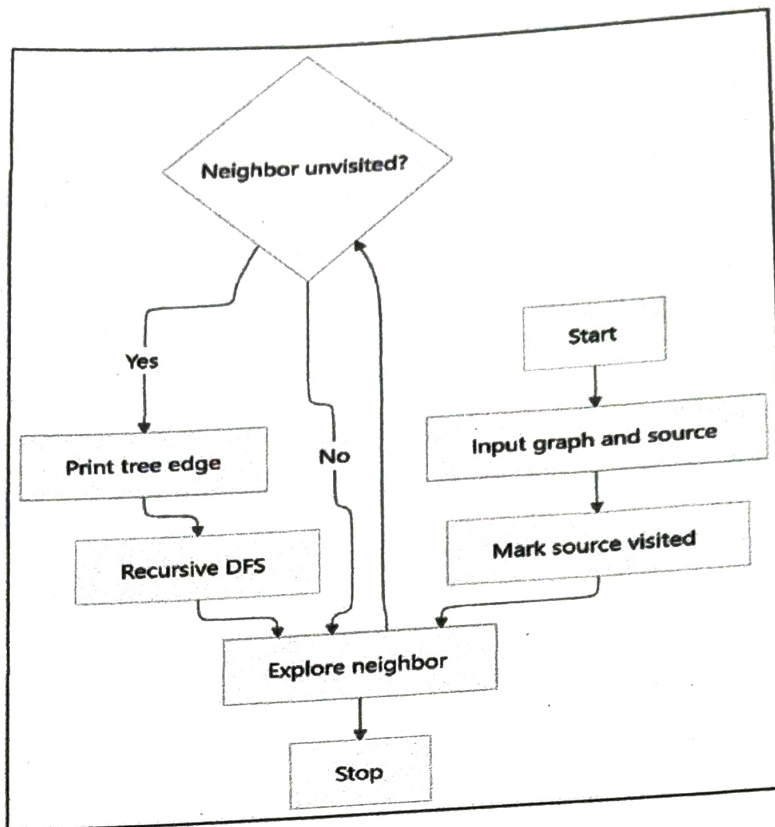
Flowchart Diagram:



4. Convert the flowchart for generating a DFS tree into code.

(20 Marks)

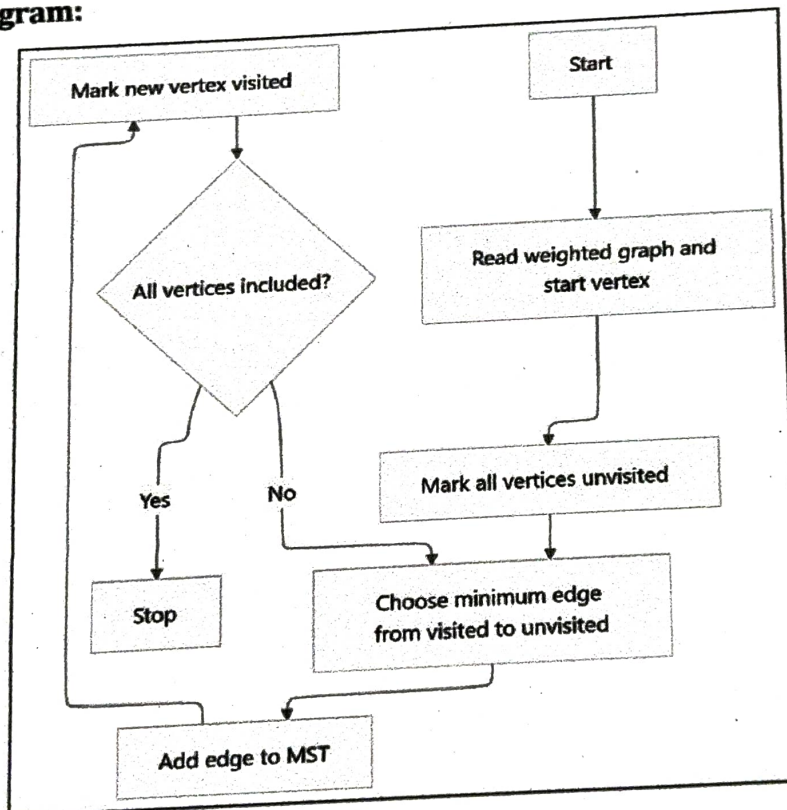
Flowchart Diagram:



(20 Marks)

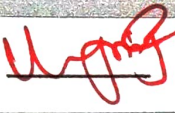
5. Convert the flowchart for Prim's algorithm into code.

Flowchart Diagram:



Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Problem Understanding	20	Demonstrates complete and clear understanding; handles edge cases effectively	Good understanding; minor gaps in edge case handling	Partial understanding; misses some requirements	Misinterprets problem; major gaps
Code Correctness	30	Fully correct; passes all test cases	Mostly correct; minor errors	Partially correct; several test cases fail	Incorrect or non-functional code
Code Quality & Readability	20	Clean, modular, well-commented, follows standards	Readable with some structure	Limited readability; poor structure	Unorganized and hard to read
Complexity Analysis	20	Accurate time & space complexity with justification	Correct but limited explanation	Incomplete or partially correct	Missing or incorrect
Testing & Validation	10	Thorough testing including edge cases	Adequate testing with few edge cases	Minimal testing	No testing evidence

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

C05 - Assessment Tool-02

- Flowchart to code conversion

Name:

Reg No:

1

Breadth First Search traversal:

```
def bfsc(graph, start):  
    visited = [False] * len(graph)  
    queue = [start]  
    visited[start] = True  
    while queue:  
        u = queue.pop(0)  
        print(u, end = " ")  
        for v in graph[u]:  
            visited[v] = True  
            queue.append(v)  
graph = {0: [1, 2], 1: [3], 2: [4, 5], 3: [], 4: [], 5: [7]}  
bfs(graph, 0)  
bfs(graph, 0)
```

2.

def kruskal(n, edges):

parent = list(range(n))

def find(x):

while parent[x] != x: x = parent[x]
return x

```

mst = [ ]
for u, v, w in sorted(edges, key = (lambda x: x[2])):
    if find(u) != find(v):
        mst.append((u, v, w))
        parent[find(u)] = find(v)
return mst

edges = [(0, 1, 10), (0, 2, 6), (0, 3, 5), (1, 3, 15), (2, 3, 4)]

```

3.

```

def topo-sort(graph, n):
    indeg = [0] * n
    for u in graph:
        for v in graph[u]:
            indeg[v] += 1
    q = [i for i in range(n) if indeg[i] == 0]
    while q:
        u = q.pop(0)
        print(u, end = " ")
        for v in graph[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    graph = {0: [1, 2], 1: [3], 2: [3], 3: []}
    topo-sort(graph, 4)

```

4.

DFS tree:

```
def dfs (graph, u, visited)
    visited [u] = True
    print (u, end = ' ')

    for v in graph [u]:
        if not visited [v]:
            dfs (graph, v, visited)

graph = {0: [1, 2], 1: [3], 3: [4, 5], 2: [7], 4: [7], 5: [7]}
visited = [False] * 6.
dfs (graph, 0, visited)
```

5.

```
def prim (graph, start):
    n = len (graph)
    visited = [False] * n
    visited [start] = True
    mst = []

    for _ in range (n-1):
        u = v = 1, w = float ('inf')
        for i in range (n):
            if not visited [i]:
                for j, c in graph [i]:
                    if not visited [j] and c < w:
                        u, v, w = i, j, c
        visited [u] = True
        mst.append (u, v, w)
```

graph = {

0: [(1,10), (2,6), (3,5)], 1: [(0,10), (3,15)]

2: [(10,6), (3,4)], 3: [(10,5), (11,15), (2,4)]

ending